

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GREEN UNIVERSITY OF BANGLADESH

Convolutional Neural Networks

Chapter 7 — A Complete Study Guide

Sharifur Rahman

*Lecturer, Department of Computer Science & Engineering
Green University of Bangladesh*

Topics Covered

- 7.1 Introduction to CNNs and Their Applications
- 7.2 Basic Idea of Convolution and Filters
- 7.3 Convolutional Layers and Feature Maps
- 7.4 Activation Functions Used in CNNs
- 7.5 Pooling Layers and Their Purpose
- 7.6 Designing a Simple CNN Architecture
- 7.7 Forward Propagation in CNNs

Contents

1	Convolutional Neural Networks	5
1.1	7.1 Introduction to Convolutional Neural Networks and Their Applications	5
1.1.1	What is a Convolutional Neural Network?	5
1.1.2	Why CNNs Instead of ANNs for Images?	5
1.1.3	Applications of CNNs	6
1.2	7.2 Basic Idea of Convolution and Filters	7
1.2.1	What is Convolution?	7
1.2.2	What is a Filter (Kernel)?	7
1.2.3	Convolution Step by Step	7
1.2.4	Important Convolution Parameters	8
1.2.5	Output Size Formula	8
1.3	7.3 Convolutional Layers and Feature Maps	9
1.3.1	The Convolutional Layer	9
1.3.2	Multiple Filters Producing Multiple Feature Maps	9
1.3.3	Hierarchical Feature Learning	9
1.3.4	Receptive Field	10
1.4	7.4 Activation Functions Used in CNNs	10
1.4.1	Why Activation Functions Are Necessary	10
1.4.2	ReLU (Rectified Linear Unit)	11
1.4.3	Leaky ReLU	11
1.4.4	Sigmoid	12
1.4.5	Softmax	12
1.5	7.5 Pooling Layers and Their Purpose	12
1.5.1	What is Pooling?	13
1.5.2	Max Pooling	13
1.5.3	Average Pooling	13
1.5.4	Why Pooling is Valuable	14
1.6	7.6 Designing a Simple CNN Architecture	14
1.6.1	Principles of CNN Architecture Design	14
1.6.2	A Simple CNN for Image Classification	15
1.6.3	Layer-by-Layer Description	15
1.6.4	Keras Implementation	15
1.7	7.7 Forward Propagation in CNNs	17
1.7.1	Overview	17
1.7.2	Step-by-Step Forward Propagation	17
1.7.3	Data Dimensions Through the Network	18
1.7.4	The Role of Forward Propagation in Training	18

Chapter 1

Convolutional Neural Networks

1.1 7.1 Introduction to Convolutional Neural Networks and Their Applications

1.1.1 What is a Convolutional Neural Network?

A **Convolutional Neural Network (CNN)** is a specialized type of deep neural network designed primarily to process data that has a grid-like structure — most commonly images and videos. Unlike traditional Artificial Neural Networks (ANNs) that treat each input value independently, CNNs exploit the **spatial structure** of images: nearby pixels are related, patterns repeat across different positions, and meaningful features exist at multiple scales.

The architecture of CNNs is inspired by the human visual system. When you look at a photograph, your brain does not examine each pixel in isolation — it detects edges first, then shapes, then objects. CNNs mimic this hierarchy by stacking layers that each learn progressively more complex features.

Definition: CNN

A **Convolutional Neural Network** is a deep learning model that uses learnable convolutional filters to automatically extract spatial features from input data, enabling it to recognize patterns regardless of their position in the input.

1.1.2 Why CNNs Instead of ANNs for Images?

Traditional ANNs fail at image tasks for three fundamental reasons:

Problem	Explanation	CNN Solution
Computational Complexity	A 224×224 RGB image has 150,528 pixels; an ANN first layer would need millions of weights	Convolutional filters share weights across the entire image
Loss of Spatial Structure	Flattening an image to a 1D vector destroys all neighbourhood relationships	Convolution operates on 2D grids, preserving spatial relationships
Location Sensitivity	ANNs must relearn the same pattern at every possible position	Convolution is translation equivariant; pooling adds translation invariance

1.1.3 Applications of CNNs

CNNs are at the heart of most modern computer vision systems. The table below lists major real-world applications:

Application	Domain	Example Use Cases
Image Classification		Identifying objects in photos (ImageNet, Google Lens)
Object Detection		Locating and labelling multiple objects in a scene (YOLO, Faster R-CNN used in autonomous vehicles)
Medical Imaging		Detecting cancer from mammograms, diabetic retinopathy from retinal scans (Google Health)
Face Recognition		Unlocking smartphones, security surveillance (DeepFace, FaceNet)
Autonomous Driving		Real-time detection of pedestrians, lanes, traffic signs (Tesla Autopilot, Waymo)
Document Analysis		Handwriting recognition, document classification (OCR systems)
Satellite Imagery		Crop monitoring, disaster assessment, urban planning
Quality Control		Detecting manufacturing defects on production lines

Real-World Example: COVID-19 Detection from Chest X-Rays

During the COVID-19 pandemic, researchers trained CNNs on chest X-ray and CT scan images to automatically detect signs of COVID-19 pneumonia. The CNN learned to identify characteristic ground-glass opacities and bilateral infiltrates that distinguish COVID-19 from other lung conditions. These models, trained on datasets of tens of thousands of images, achieved diagnostic accuracy comparable to experienced radiologists and could process each scan in under a second — far faster

than manual review. This application demonstrated how CNNs can be rapidly deployed in public health emergencies to assist overwhelmed medical systems.

Key Point

Key insight: CNNs learn *what features to look for* directly from labelled data. No human expert needs to manually design feature detectors. The network discovers, entirely on its own, that edges, textures, and shapes are useful for classification.

1.2 7.2 Basic Idea of Convolution and Filters

1.2.1 What is Convolution?

Convolution is the core mathematical operation in a CNN. At its simplest, convolution involves sliding a small grid of numbers — called a **filter** or **kernel** — across the entire input image and computing a weighted sum at every position. The result is a new grid of numbers called a **feature map** that highlights where in the image the pattern defined by the filter was found.

Definition: Convolution Operation

Given an input image I and a filter (kernel) K , the convolution output at position (i, j) is:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

The filter K slides across I with a step called the **stride**, computing a dot product at each position.

1.2.2 What is a Filter (Kernel)?

A **filter** is a small matrix of numbers, typically 3×3 , 5×5 , or 7×7 . Each filter is designed (or learned) to detect a specific visual pattern. When the filter is placed over a region of the image that matches its pattern, the dot product produces a high value. Over regions that do not match, the output is low or negative.

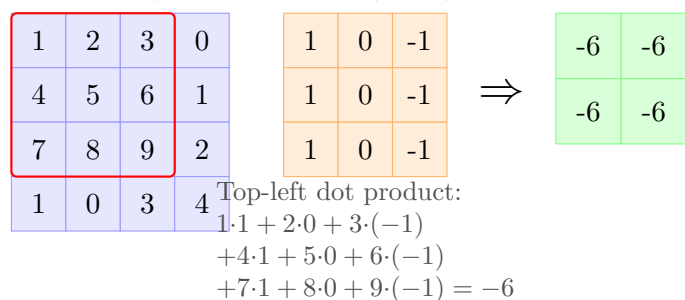
Key properties of filters:

- Filters are **learnable** — their values are initialized randomly and refined through training via backpropagation.
- The same filter is applied to every position in the image (**weight sharing**), drastically reducing the number of parameters.
- Multiple different filters are applied to the same image simultaneously, each detecting a different pattern.

1.2.3 Convolution Step by Step

Consider a simple 4×4 grayscale image and a 3×3 filter:

Input Image (4×4) Filter (3×3) Feature Map (2×2)



1.2.4 Important Convolution Parameters

Parameter	Meaning	Effect
Filter size	Dimensions of the kernel (e.g. 3×3)	Larger filters capture broader patterns; small filters capture fine details
Stride	How many pixels the filter moves at each step	Larger stride reduces output size; stride 1 is most common
Padding	Adding zeros around the image border	“Same” padding preserves input dimensions; “Valid” padding reduces them
Number of filters	How many different filters are applied	More filters = more features detected at each layer

1.2.5 Output Size Formula

Formula

Given input size $W \times H$, filter size $F \times F$, padding P , and stride S :

$$\text{Output Width} = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$$

$$\text{Output Height} = \left\lfloor \frac{H - F + 2P}{S} \right\rfloor + 1$$

Example: Edge Detection Filter

One of the simplest and most important filters is the vertical edge detector:

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad (\text{Sobel Filter})$$

When this filter slides over an image, positions where pixel values change sharply from left to right (vertical edges) produce high output values. Flat, uniform regions

produce values near zero. This exact type of filter is learned automatically by the first convolutional layer of a CNN during training. In a face recognition system, early layers learn filters for detecting edges of the nose, eyes, and mouth outlines.

Key Point

Weight sharing advantage: A 3×3 filter applied to a 1000×1000 image uses only 9 parameters regardless of image size. An equivalent fully connected layer would require $1,000,000 \times 1,000,000 = 10^{12}$ parameters — computationally impossible.

1.3 7.3 Convolutional Layers and Feature Maps

1.3.1 The Convolutional Layer

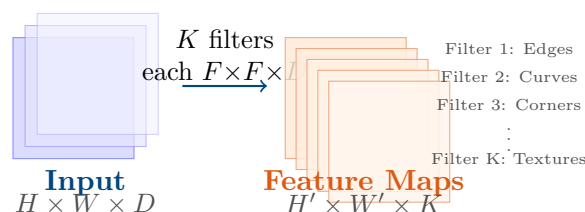
A **convolutional layer** is the fundamental building block of a CNN. It contains a collection of learnable filters, each of which produces one output feature map. All filters in the same layer operate on the same input and produce independent feature maps that are stacked together to form the layer's output volume.

Definition: Feature Map

A **feature map** (also called an activation map) is the output produced by sliding a single filter across the input. Each value in the feature map indicates the strength of the filter's detected pattern at the corresponding spatial location in the input.

1.3.2 Multiple Filters Producing Multiple Feature Maps

In practice, a convolutional layer applies dozens or hundreds of filters simultaneously. If layer ℓ applies K filters to an input of depth D , the output is a 3D volume with depth K :



1.3.3 Hierarchical Feature Learning

One of the most powerful properties of CNNs is that stacking multiple convolutional layers causes the network to automatically learn a **hierarchy of features**:

Layer Depth	Features Detected	Example (Face Recognition)
Layer 1	Edges, lines, colour gradients	Horizontal and vertical boundaries
Layer 2	Textures, corners, simple shapes	Eye corners, lip edges, skin texture
Layer 3	Object parts	Eyes, nose tip, ear shapes
Layer 4+	Whole objects or semantic concepts	Full faces, facial expressions

1.3.4 Receptive Field

The **receptive field** of a neuron in layer ℓ is the region of the original input image that influences its activation. As depth increases, receptive fields grow, allowing deeper neurons to respond to larger, more complex patterns.

Example: Feature Maps in a CNN Trained on CIFAR-10

Consider a CNN trained to classify images from the CIFAR-10 dataset (10 categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck).

After the first convolutional layer (32 filters, 3×3 kernel), 32 feature maps are produced. Each map responds to a different low-level characteristic of the input image: some activate strongly along diagonal edges, others along horizontal lines, and others respond to specific colour transitions.

After the second convolutional layer, the feature maps respond to higher-level combinations: a filter might activate when it detects a pattern of edges that forms an elongated shape characteristic of an airplane wing, or a circular arrangement characteristic of a wheel. By the time the image reaches the fully connected layer, these feature maps have condensed the entire $32 \times 32 \times 3$ input into a compact representation that makes the 10-class classification straightforward.

Key Point

Depth vs Width: Adding more filters per layer (width) increases the variety of features detected at that level. Adding more layers (depth) increases the abstraction and complexity of features the network can recognize. Modern architectures like ResNet-50 use 50 layers and hundreds of filters per layer.

1.4 7.4 Activation Functions Used in CNNs

1.4.1 Why Activation Functions Are Necessary

Convolution is a **linear operation**: it computes weighted sums of inputs. A network built entirely of linear operations — no matter how deep — can only learn linear transformations, which are far too simple to model the complex patterns found in real images. **Activation functions** introduce non-linearity, enabling the network to learn arbitrarily complex mappings from inputs to outputs.

After each convolutional layer, an activation function is applied element-wise to every value in the output feature maps before passing to the next layer.

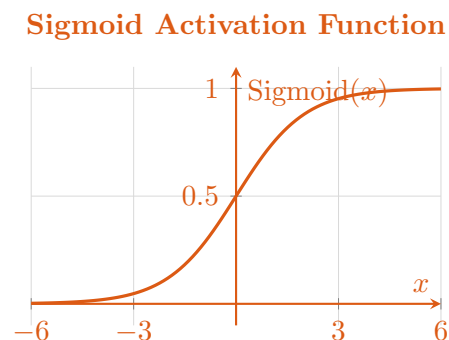
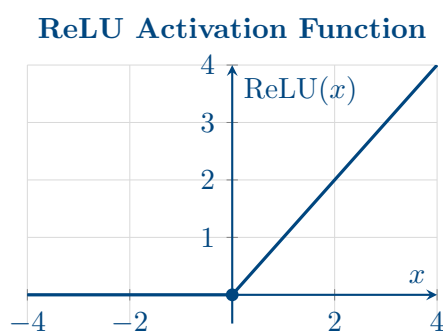
1.4.2 ReLU (Rectified Linear Unit)

ReLU is by far the most widely used activation function in CNNs:

Formula

$$\text{ReLU}(x) = \max(0, x)$$

Negative values are set to zero; positive values pass through unchanged.



Why ReLU dominates in CNNs:

- **Computationally trivial:** Just a threshold at zero — much faster than sigmoid or tanh.
- **No vanishing gradient for positive inputs:** The gradient is exactly 1 for $x > 0$, allowing gradients to flow unchanged through many layers.
- **Sparsity:** Many neurons output zero, creating sparse representations that are computationally efficient and reduce overfitting.
- **Biologically plausible:** Real neurons are either firing or not; ReLU approximates this binary behaviour.

1.4.3 Leaky ReLU

Plain ReLU has a “dying neurons” problem: neurons whose inputs are always negative receive zero gradient and stop learning. Leaky ReLU solves this:

Formula

$$\text{Leaky ReLU}(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases} \quad \text{where typically } \alpha = 0.01$$

1.4.4 Sigmoid

Formula

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \text{output range: } (0, 1)$$

Sigmoid squashes any input to a probability between 0 and 1. It is not used in hidden CNN layers (suffers from vanishing gradients in deep networks) but is used in the final output layer for binary classification.

1.4.5 Softmax

Used exclusively in the **output layer** for multi-class classification:

Formula

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

Converts raw scores (logits) from the final layer into a probability distribution over C classes. All outputs sum to exactly 1.

Function	Formula	Output Range	Where Used in CNN
ReLU	$\max(0, x)$	$[0, +\infty)$	All hidden conv layers
Leaky ReLU	$\max(\alpha x, x)$	$(-\infty, +\infty)$	Hidden layers (alternative)
Sigmoid	$1/(1 + e^{-x})$	$(0, 1)$	Binary classification output
Softmax	$e^{z_i} / \sum e^{z_j}$	$(0, 1)$, sum= 1	Multi-class output layer

Example: Activation in a Cat vs Dog Classifier

A CNN trained to classify cats and dogs processes a cat image through its convolutional layers. After each convolution, ReLU is applied:

Before ReLU (sample values from one filter): $[-2.3, 0.8, -0.1, 4.2, -1.5, 2.1]$

After ReLU: $[0, 0.8, 0, 4.2, 0, 2.1]$

The negative values — regions where the filter pattern was not detected — are suppressed to zero. Only the strong positive activations (where cat-relevant features like pointed ears or fur textures were found) are passed forward. This creates a sparse, informative representation that makes subsequent layers' work much easier. In the final output layer, Softmax converts the two raw scores into, for example, $[\text{cat} : 0.92, \text{dog} : 0.08]$ — a confident cat classification.

1.5 7.5 Pooling Layers and Their Purpose

1.5.1 What is Pooling?

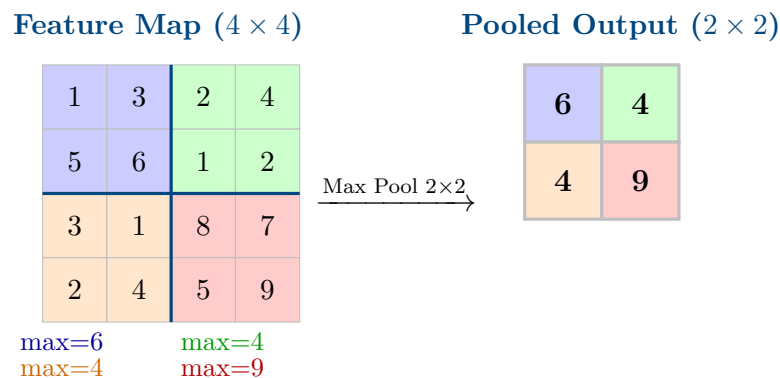
After convolution and activation, the feature maps can still be quite large. **Pooling layers** reduce the spatial dimensions (height and width) of feature maps, keeping only the most important information from each region. This is also called **subsampling** or **downsampling**.

Definition: Pooling Layer

A **pooling layer** applies a summarizing operation to non-overlapping (or slightly overlapping) rectangular regions of the feature map, replacing each region with a single representative value.

1.5.2 Max Pooling

Max pooling is the most common pooling operation. It takes a rectangular window and outputs the maximum value within that window:



1.5.3 Average Pooling

Average pooling computes the mean of all values within the window instead of the maximum. It produces smoother feature maps and is used in some architectures (notably GoogLeNet uses Global Average Pooling to replace fully connected layers).

Pooling Type		Operation	When to Use
Max Pooling		Takes the maximum value in each window	Most common; best for detecting “is this feature present?”
Average Pooling		Takes the mean value in each window	When overall distribution matters more than peak activation
Global Max Pooling	Max	Single maximum over entire feature map	Compact representation for small datasets
Global Average Pooling	Average	Single average over entire feature map	Final layer before classification in modern architectures

1.5.4 Why Pooling is Valuable

Pooling serves four important purposes:

- 1. Dimensionality Reduction:** A 2×2 max pooling with stride 2 reduces each feature map to one quarter of its original size. This reduces computation and memory requirements for all subsequent layers.
- 2. Translation Invariance:** If a feature (e.g. a cat's eye) shifts slightly in the image, it will still fall within the same pooling window and produce the same maximum output. This makes the network robust to small positional shifts in the input.
- 3. Overfitting Control:** Fewer parameters in subsequent layers means less opportunity for overfitting to training data specifics.
- 4. Increasing Receptive Field:** After pooling, each neuron in the next layer effectively “sees” a larger portion of the original image, enabling detection of larger-scale patterns.

Example: Pooling in Autonomous Driving

In an autonomous vehicle, the on-board CNN must detect pedestrians in real time from the car's cameras. When a pedestrian is photographed, their exact pixel position shifts slightly between consecutive frames as the car moves. Without pooling, small positional shifts would produce completely different feature map values, potentially causing the network to miss the pedestrian.

Max pooling applied after each convolutional block ensures that even if the pedestrian's leg appears one or two pixels to the left compared to the training image, the maximum activation within the pooling window remains similar. This **spatial invariance** is critical for safe, reliable object detection in dynamic environments. Systems like those used in Tesla Autopilot and Waymo rely on this property.

Key Point

Modern trend: Some recent architectures (e.g. fully convolutional networks, vision transformers) reduce or eliminate pooling, instead using strided convolutions to achieve downsampling. However, max pooling remains the standard in classical CNN architectures and is essential to understand.

1.6 7.6 Designing a Simple CNN Architecture

1.6.1 Principles of CNN Architecture Design

Designing a CNN architecture means deciding how to stack convolutional layers, pooling layers, and fully connected layers to solve a specific task. Good architecture design follows a set of well-established principles.

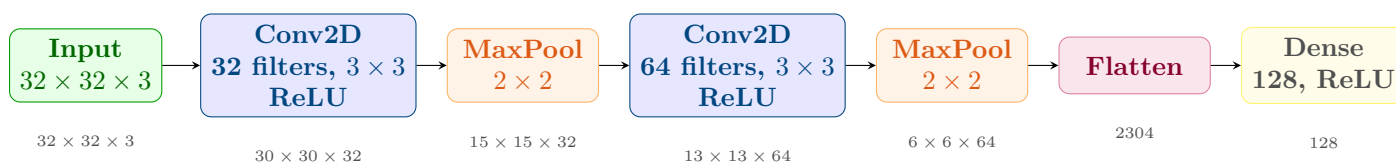
Principle 1 — Feature Extraction then Classification: Every CNN has two conceptually distinct phases. The first, consisting of convolutional and pooling layers, extracts increasingly abstract features from the raw input. The second, consisting of fully connected layers, uses those features to make the final prediction.

Principle 2 — Spatial Dimensions Decrease, Depth Increases: As the network goes deeper, height and width of feature maps decrease (due to pooling or stride), but the number of feature maps (channels) increases. Deeper layers represent more abstract concepts that require more channels to capture the increasing variety of patterns.

Principle 3 — Start Small: For a new problem, begin with the simplest architecture that could plausibly work: one or two convolutional blocks, modest filter counts, and a small fully connected section. Add complexity only if the simple model underfits.

1.6.2 A Simple CNN for Image Classification

Below is a canonical simple CNN architecture for classifying 32×32 RGB images into 10 classes (such as CIFAR-10):



1.6.3 Layer-by-Layer Description

Layer	Output Shape	Purpose
Input	$32 \times 32 \times 3$	Normalized RGB pixel values
Conv2D (32, 3×3 , ReLU)	$30 \times 30 \times 32$	Detect 32 types of low-level features (edges, colours)
MaxPool2D (2×2)	$15 \times 15 \times 32$	Halve spatial size; add translation invariance
Conv2D (64, 3×3 , ReLU)	$13 \times 13 \times 64$	Detect 64 mid-level features (textures, shapes)
MaxPool2D (2×2)	$6 \times 6 \times 64$	Halve again; further abstraction
Flatten	2304	Convert 3D volume to 1D vector for dense layers
Dense (128, ReLU)	128	Combine learned features; learn classification logic
Dense (10, Soft-max)	10	Output class probabilities for 10 categories

1.6.4 Keras Implementation

The architecture above is implemented in Keras as follows:

```

1 import tensorflow as tf
2 from tensorflow.keras import layers, models
3
4 # Build a simple CNN for 10-class image classification
5 cnn = models.Sequential([
6
7     # Block 1: First Convolutional Block
  
```

```
8      # 32 filters of size 3x3, ReLU activation
9      # Input: 32x32 RGB images
10     layers.Conv2D(32, (3, 3), activation='relu', input_shape
11                    =(32, 32, 3)),
12
13     # Max pooling: reduces 30x30 -> 15x15
14     layers.MaxPooling2D((2, 2)),
15
16     # Block 2: Second Convolutional Block
17     # 64 filters to capture more complex features
18     layers.Conv2D(64, (3, 3), activation='relu'),
19
20     # Max pooling: reduces 13x13 -> 6x6
21     layers.MaxPooling2D((2, 2)),
22
23     # Transition: Flatten 3D feature maps to 1D vector
24     layers.Flatten(),           # 6 x 6 x 64 = 2304 values
25
26     # Classification Head
27     layers.Dense(128, activation='relu'), # Learn to combine
28                                           features
29
30     # Output: 10 neurons, one per class, Softmax for
31     # probabilities
32     layers.Dense(10, activation='softmax')
33 ])
```

```
34 # Compile with Adam optimizer and cross-entropy loss
35 cnn.compile(
36     optimizer='adam',
37     loss='sparse_categorical_crossentropy',
38     metrics=['accuracy']
39 )
40
41 # Display the architecture summary
42 cnn.summary()
```

Example: Architecture Decisions for Handwritten Digit Recognition

For the MNIST dataset (handwritten digits 0–9, 28×28 grayscale images), a simple CNN design might be:

Input: $28 \times 28 \times 1$ (single channel, no RGB)

Conv2D(32, 3x3, ReLU): Detects edges and strokes — output $26 \times 26 \times 32$

MaxPool2D(2x2): Reduces to $13 \times 13 \times 32$

Conv2D(64, 3x3, ReLU): Detects digit sub-components (curves, loops) — output $11 \times 11 \times 64$

MaxPool2D(2x2): Reduces to $5 \times 5 \times 64$

Flatten: $5 \times 5 \times 64 = 1600$ values

Dense(64, ReLU): Learns combinations of features for each digit

Dense(10, Softmax): Outputs probability for digits 0 through 9

This architecture achieves over 99% accuracy on MNIST, outperforming any ANN baseline while using fewer parameters thanks to weight sharing.

1.7 7.7 Forward Propagation in CNNs

1.7.1 Overview

Forward propagation is the process of passing an input image through every layer of the CNN in sequence — from input to output — to produce a prediction. At each layer, the input is transformed by that layer’s operation and the result is passed to the next layer. No weights are updated during forward propagation; this is purely a *compute the answer* step.

Forward propagation must be executed for every training example (to compute the loss) and for every test or real-world input (to produce a prediction).

1.7.2 Step-by-Step Forward Propagation

Consider passing a single image through the simple CNN defined in Section 7.6. Each stage transforms the data as follows:

Step 1 — Input Normalization: Raw pixel values (0–255) are scaled to the range $[0, 1]$ by dividing by 255. This ensures all inputs are on the same scale, which makes gradient descent behave more stably.

Step 2 — Convolutional Layer: Each of the 32 filters performs the convolution operation: the filter slides across the input, computing a dot product at every position. The result is 32 feature maps, each highlighting where in the image that filter’s pattern was found. This step is the most computationally intensive in the network.

Step 3 — Activation (ReLU): The activation function is applied element-wise to every value in every feature map. Negative values become zero; positive values are unchanged. The feature maps are now *activation maps*.

Step 4 — Pooling: Max pooling reduces each feature map’s spatial dimensions. From 32 maps of size 30×30 , the output is 32 maps of size 15×15 . The most prominent activation in each 2×2 region is retained.

Step 5 — Repeat for Layer 2: Steps 2–4 repeat for the second convolutional block (64 filters), producing 64 feature maps of size 6×6 .

Step 6 — Flatten: The 3D volume ($6 \times 6 \times 64$) is reshaped into a 1D vector of length 2304. This bridges the spatial feature extraction phase and the classification phase.

Step 7 — Fully Connected Layer: The flattened vector is multiplied by the dense layer’s weight matrix (shape 2304×128) and a bias is added. ReLU activation is applied, producing a 128-dimensional representation.

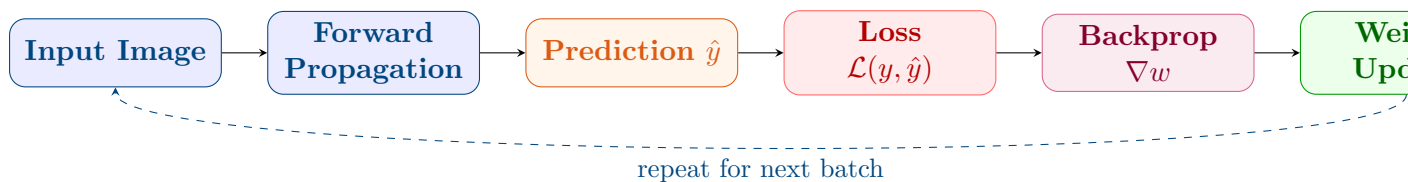
Step 8 — Output Layer + Softmax: The final dense layer maps 128 values to 10 raw scores (one per class). Softmax converts these into a probability distribution. The predicted class is the one with the highest probability.

1.7.3 Data Dimensions Through the Network

Layer	Output Shape	What Happens
Input (normalized)	$32 \times 32 \times 3$	Pixel values in $[0, 1]$
Conv2D Layer 1	$30 \times 30 \times 32$	32 filters applied; each produces a feature map
ReLU Activation	$30 \times 30 \times 32$	Negative values zeroed out
MaxPool Layer 1	$15 \times 15 \times 32$	2×2 max pooling; spatial dimensions halved
Conv2D Layer 2	$13 \times 13 \times 64$	64 filters detect more complex features
ReLU Activation	$13 \times 13 \times 64$	Non-linearity applied again
MaxPool Layer 2	$6 \times 6 \times 64$	Spatial dimensions halved again
Flatten	2304	3D volume reshaped to 1D
Dense Layer (ReLU)	128	Weighted combination of all features
Output (Softmax)	10	Class probability distribution

1.7.4 The Role of Forward Propagation in Training

Forward propagation is the first half of every training iteration. After the forward pass produces predictions, the **loss function** measures how wrong those predictions are. The loss value is then used by **backpropagation** to compute gradients, and finally the **optimizer** updates the weights. This cycle — forward pass, loss computation, backward pass, weight update — repeats for every mini-batch throughout training.



Example: Tracing One Image Through the Network

An image of a cat from the CIFAR-10 dataset ($32 \times 32 \times 3$) is passed through the CNN:

After normalization: Pixel values converted from integers $[0, 255]$ to floats $[0, 1]$.

After Conv2D Layer 1 (32 filters, 3×3): 32 feature maps of size 30×30 . Filter 7 has learned to detect orange-brown fur texture; it activates strongly over the cat's body. Filter 12 detects curved edges; it activates at the outline of the ears.

After MaxPool: 32 maps of size 15×15 . The ear-curve features are still visible, now more spatially compact.

After Conv2D Layer 2 (64 filters): 64 maps of size 13×13 . Some filters now respond to combinations of features: "curved edge + fur texture nearby" — a pattern highly indicative of cat body parts.

After second MaxPool: 64 maps of 6×6 . The spatial detail is greatly reduced, but the presence or absence of cat-relevant features is strongly encoded in the channel dimension.

After Flatten and Dense layers: A 128-dimensional vector captures the overall profile of the image’s feature content.

Softmax output: [airplane : 0.01, automobile : 0.01, bird : 0.04, **cat:0.85**, deer : 0.03, ...]

The network correctly predicts “cat” with 85% confidence.

Key Point

Inference vs Training: During *training*, forward propagation is followed by loss computation and backpropagation. During *inference* (real-world use), only the forward pass is performed — the network simply computes its answer. This is why trained models can make predictions in milliseconds.

Chapter 7 — Complete Summary

- **7.1 Introduction:** CNNs are deep networks designed for grid-structured data like images. They overcome ANNs’ weaknesses (spatial blindness, computational excess, location sensitivity) and power applications from medical imaging to autonomous vehicles.
- **7.2 Convolution and Filters:** Convolution slides a small learnable filter across the image, computing dot products at each position. Weight sharing makes this vastly more efficient than fully connected layers. Different filters detect different patterns.
- **7.3 Feature Maps:** Each filter produces one feature map showing where its pattern occurs in the input. Stacking multiple convolutional layers creates a hierarchy: early layers detect edges, middle layers detect shapes, deep layers detect objects.
- **7.4 Activation Functions:** ReLU ($\max(0, x)$) is the standard activation for CNN hidden layers. It is fast, avoids vanishing gradients, and creates sparsity. Softmax is used at the output for multi-class classification.
- **7.5 Pooling:** Max pooling reduces spatial dimensions, retaining the strongest activations. It decreases computation, provides translation invariance, and controls overfitting.
- **7.6 Architecture Design:** A CNN has a feature extraction phase (conv + pool layers, spatial dimensions shrink, depth grows) followed by a classification phase (flatten + dense layers). Start simple and add complexity only when needed.
- **7.7 Forward Propagation:** An image passes sequentially through every layer — convolution, activation, pooling, flatten, dense — producing a class probability vector. This is the prediction step; weight updates occur only during the subsequent backpropagation step.